

NumPy

Numerical Python — The Foundation of Scientific Computing

AI/ML Engineer Learning Series • Deep Dive Edition

Topic	NumPy
Full Name	Numerical Python
Current Version	2.x
Core Strength	Vectorised N-dimensional array operations
Why It Matters	Foundation of Pandas, Scikit-learn, TensorFlow, PyTorch

What's Inside

Here is everything covered in this guide, in order:

#	Section	What you'll learn
1	What Is NumPy?	Why it exists, list vs array
2	The ndarray	The core object, attributes, dtypes
3	Creating Arrays	All the ways to make arrays
4	Indexing & Slicing	Picking elements, masks, fancy indexing
5	Reshaping	Changing shape, stacking, splitting
6	Broadcasting	Math on different-shaped arrays
7	Math & Aggregations	ufuncs, axis, linear algebra
8	Performance	Why NumPy is fast, vectorisation
9	Gotchas	Common mistakes and edge cases
10	Real ML Patterns	Normalisation, regression, softmax
★	Cheatsheet	Quick reference of every command

1. What Is NumPy?

NumPy (Numerical Python) is an open-source library that provides support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions. At its core is the **ndarray** — a fixed-type, contiguous block of memory that is orders of magnitude faster than Python lists for numerical work.

Almost every major Python ML/data library is built on top of NumPy. Pandas wraps it, Scikit-learn uses it internally, TensorFlow and PyTorch mirror its API, and Matplotlib expects NumPy arrays. Understanding NumPy deeply means you understand the substrate everything else sits on.

Why Not Just Use Python Lists?

Feature	Python List	NumPy ndarray
Type enforcement	Any mix of types	Single dtype (homogeneous)
Memory layout	Pointers to objects (scattered)	Contiguous block (cache-friendly)
Speed	Interpreted loop, slow	C/Fortran vectorised, very fast
Broadcasting	Manual loops required	Built-in, zero-copy
Memory usage	High (object overhead)	Low (raw bytes)
N-dimensional	Nested lists (awkward)	Native nd-arrays

■ *Rule of thumb: if you're writing a Python for-loop over numbers, NumPy can probably do it 10–100× faster.*

2. The ndarray — Core Data Structure

The **ndarray** stores data as a flat contiguous block of bytes in memory, described by metadata: shape, dtype, strides, and a data pointer. Shape tells you how many dimensions and how many elements per dimension. Strides tell NumPy how many bytes to skip to reach the next element along each axis.

Contiguous Memory Block — 1D Array of shape (9,)

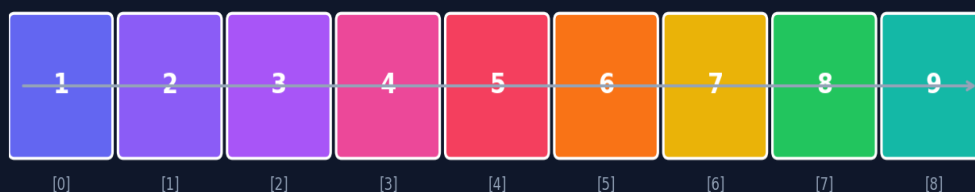


Fig 1 — A 1D ndarray stored as a contiguous block of memory with indexed slots.

Key Attributes

Attribute	Description	Example output
arr.shape	Tuple of dimension sizes	(3, 4)

arr.dtype	Data type of elements	float64
arr.ndim	Number of dimensions	2
arr.size	Total number of elements	12
arr.nbytes	Total memory in bytes	96
arr.strides	Bytes to step per axis	(32, 8)
arr.T	Transposed view (no copy)	shape (4, 3)
arr.flat	1D iterator over elements	—

Data Types (dtype)

Choosing the right dtype saves memory and increases speed. Default is float64 for floats and int64 for integers on most platforms.

dtype	Size	Range / Precision	When to use
int8	1 byte	-128 to 127	Pixel labels, tiny counts
int32	4 bytes	~-2B to 2B	General integer data
int64	8 bytes	Very large integers	Default int on 64-bit
float32	4 bytes	~7 decimal digits	ML model weights (GPU)
float64	8 bytes	~15 decimal digits	Scientific computation
bool	1 byte	True / False	Masks, conditions
complex128	16 bytes	Complex numbers	Signal processing, FFT

3. Creating Arrays

From Python Data

```
import numpy as np

# From list
a = np.array([1, 2, 3])          # shape (3,)
b = np.array([[1,2],[3,4]], dtype=float) # shape (2,2), float64

# Check
print(a.shape, a.dtype)  # (3,) int64
print(b.shape, b.dtype)  # (2,2) float64
```

Built-in Constructors

Function	Output	Use case
<code>np.zeros((3,4))</code>	3×4 array of 0.0	Initialise weight matrices
<code>np.ones((2,3))</code>	2×3 array of 1.0	Initialise bias terms
<code>np.full((3,3), 7)</code>	3×3 filled with 7	Constant arrays
<code>np.eye(4)</code>	4×4 identity matrix	Linear algebra
<code>np.arange(0,10,2)</code>	[0,2,4,6,8]	Like Python range()
<code>np.linspace(0,1,5)</code>	[0, .25, .5, .75, 1]	Evenly spaced floats
<code>np.random.rand(3,3)</code>	Uniform [0,1) values	Random initialisation
<code>np.random.randn(3,3)</code>	Standard normal values	Gaussian noise
<code>np.empty((4,4))</code>	Uninitialised memory	Speed — when you'll fill it

■ `np.empty()` is the fastest way to allocate — it skips zeroing memory. Only use it when you know you'll fill every element before reading.

Reproducible Random Arrays

```
rng = np.random.default_rng(seed=42)  # modern API (NumPy 1.17+)
X = rng.normal(0, 1, size=(100, 5))   # 100 samples, 5 features
y = rng.integers(0, 2, size=100)      # binary labels

# Old API (still common in tutorials)
np.random.seed(42)
X = np.random.randn(100, 5)
```

4. Indexing & Slicing

NumPy indexing is one of the most important skills to master. Slices return **views** (not copies) — modifying a slice modifies the original. Fancy indexing (integer/boolean arrays) returns copies.

Basic Indexing

```
arr = np.arange(12).reshape(3, 4)
# [[ 0  1  2  3]
#  [ 4  5  6  7]
#  [ 8  9 10 11]]

arr[1, 2]          # → 6   (row 1, col 2)
arr[0]             # → [0 1 2 3] (entire first row)
arr[:, 2]          # → [2 6 10] (entire third column)
arr[1:, 1:3]       # → [[5 6],[9 10]] (submatrix)
arr[-1]           # → [8 9 10 11] (last row)
arr[:, ::2, ::2]   # → [[0 2],[8 10]] (every other row/col)
```

Boolean (Mask) Indexing

```
data = np.array([10, -3, 7, -1, 5, -8])

mask = data > 0          # [True False True False True False]
data[mask]              # → [10  7  5]   — only positives

# In-place: zero out negatives
data[data < 0] = 0       # [10  0  7  0  5  0]

# Compound conditions
data[(data > 3) & (data < 9)] # → [7 5]
```

Fancy Indexing

```
arr = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
arr[idx]          # → [10 30 50] — always a COPY

# 2D fancy indexing
M = np.arange(16).reshape(4,4)
rows = [0, 2]
cols = [1, 3]
M[rows, cols]     # → [1, 11] (M[0,1] and M[2,3])
```

■ **View vs Copy gotcha:** `arr[1:3]` is a **view** — changing it changes `arr`. `arr[[1,2]]` (fancy index) is a **copy** — changes do **NOT** affect `arr`. Use `arr.copy()` explicitly when you want a true copy of a slice.

5. Reshaping & Array Manipulation

```
a = np.arange(12)

a.reshape(3, 4)          # 3 rows 4 cols – view if possible
a.reshape(2, -1)         # -1 means 'figure it out' → (2,6)
a.reshape(2, 2, 3)       # 3D array (2 blocks, 2 rows, 3 cols)

a.ravel()                # always 1D, view if possible
a.flatten()              # always 1D, always a copy

# Transpose
M = np.ones((3, 4))
M.T                      # shape (4, 3)
M.transpose(1, 0)        # same as .T for 2D

# Add/remove dimensions
v = np.array([1,2,3])    # shape (3,)
v[:, np.newaxis]         # shape (3,1) – column vector
v[np.newaxis, :]        # shape (1,3) – row vector
np.expand_dims(v, 0)     # same as above
np.squeeze(v[np.newaxis,:]) # removes size-1 dims → (3,)
```

Stacking & Splitting

```
a = np.array([[1,2],[3,4]])
b = np.array([[5,6],[7,8]])

np.vstack([a, b])        # vertical stack → shape (4,2)
np.hstack([a, b])        # horizontal stack → shape (2,4)
np.concatenate([a,b], axis=0) # same as vstack
np.stack([a, b], axis=0)   # new axis → shape (2,2,2)

# Splitting
M = np.arange(12).reshape(3,4)
np.hsplit(M, 2)          # split into 2 along cols → each (3,2)
np.vsplit(M, 3)          # split into 3 along rows → each (1,4)
```

6. Broadcasting

Broadcasting is NumPy's mechanism for doing arithmetic on arrays of different shapes without making extra copies of data. It's one of the most powerful — and most misunderstood — features.

The Broadcasting Rules

Rule	Description
1. Align shapes from the right	Compare dimensions right-to-left

2. Missing dims = size 1	A (3,) array is treated as (1,3)
3. Size-1 dims stretch	A dim of size 1 is repeated to match
4. Both must be 1 or equal	Any other mismatch raises an error

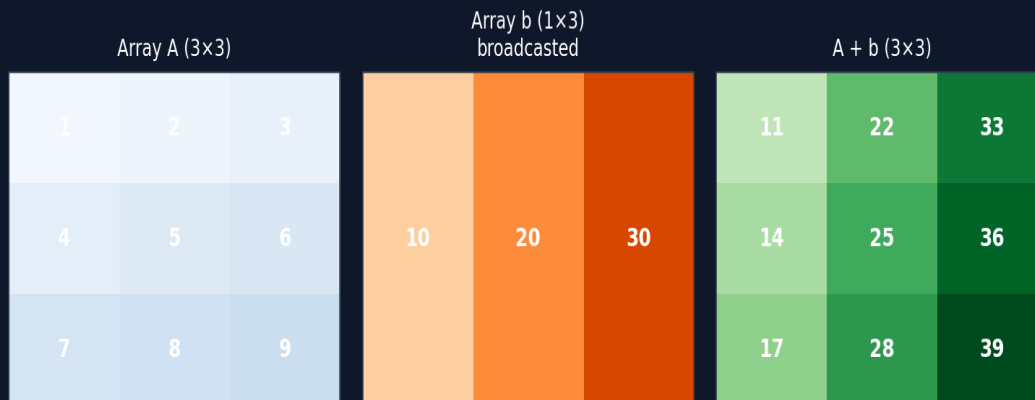


Fig 2 — Broadcasting: a (1x3) vector is stretched to (3x3) before adding to A.

```
A = np.array([[1,2,3],[4,5,6],[7,8,9]]) # (3,3)
b = np.array([10, 20, 30]) # (3,) → treated as (1,3)

A + b # b broadcast across all rows
# [[11,22,33],
#  [14,25,36],
#  [17,28,39]]

# Common ML use: centre features
X = rng.normal(size=(1000, 5))
mean = X.mean(axis=0) # shape (5,)
X_centered = X - mean # (1000,5) - (5,) → works!
```

■ **Edge case:** (3,1) + (1,4) → (3,4). Both dims stretch independently. But (3,2) + (3,3) raises `ValueError` — neither is size 1.

7. Mathematical Operations & Aggregations

Element-wise Universal Functions (ufuncs)

Ufuncs are compiled C functions that operate element-wise. They're the reason NumPy is fast — no Python interpreter overhead per element.

Category	Functions
Arithmetic	np.add, np.subtract, np.multiply, np.divide, np.power, np.mod
Trig	np.sin, np.cos, np.tan, np.arcsin, np.arctan2
Exponential/Log	np.exp, np.exp2, np.log, np.log2, np.log10
Comparison	np.greater, np.less, np.equal, np.maximum, np.minimum
Rounding	np.round, np.floor, np.ceil, np.trunc
Type	np.abs, np.sign, np.real, np.imag, np.conj

Aggregation Functions — The axis= Parameter

```
M = np.array([[1,2,3],
              [4,5,6]])

M.sum()          # 21          – sum of ALL elements
M.sum(axis=0)    # [5, 7, 9]   – sum DOWN rows (per column)
M.sum(axis=1)    # [6, 15]     – sum ACROSS cols (per row)

M.mean(axis=0)   # [2.5, 3.5, 4.5]
M.std(axis=1)    # std of each row
M.max(axis=0)    # [4, 5, 6]
M.argmax(axis=0) # index of max: [1, 1, 1]
M.cumsum(axis=1) # running sum along rows

# keepdims=True preserves shape for broadcasting
M.mean(axis=1, keepdims=True) # shape (2,1) not (2,)
```

Linear Algebra (np.linalg)

```

A = np.array([[3.,1.],[1.,2.]])
b = np.array([9., 8.])

np.linalg.det(A)          # determinant → 5.0
np.linalg.inv(A)          # matrix inverse
np.linalg.solve(A, b)     # solve Ax=b → [2. 3.]
np.linalg.norm(b)         # Euclidean norm → 12.04
np.linalg.eig(A)          # eigenvalues & eigenvectors
np.linalg.svd(A)          # singular value decomposition
np.dot(A, b)              # matrix-vector product
A @ b                     # same, cleaner syntax (Python 3.5+)

```

8. Performance — Why NumPy Is Fast

NumPy achieves its speed through: (1) **contiguous memory** enabling CPU cache locality, (2) **vectorised C/Fortran loops** via BLAS/LAPACK, (3) **zero Python overhead** per element in ufuncs, and (4) optional parallelism via MKL or OpenBLAS.

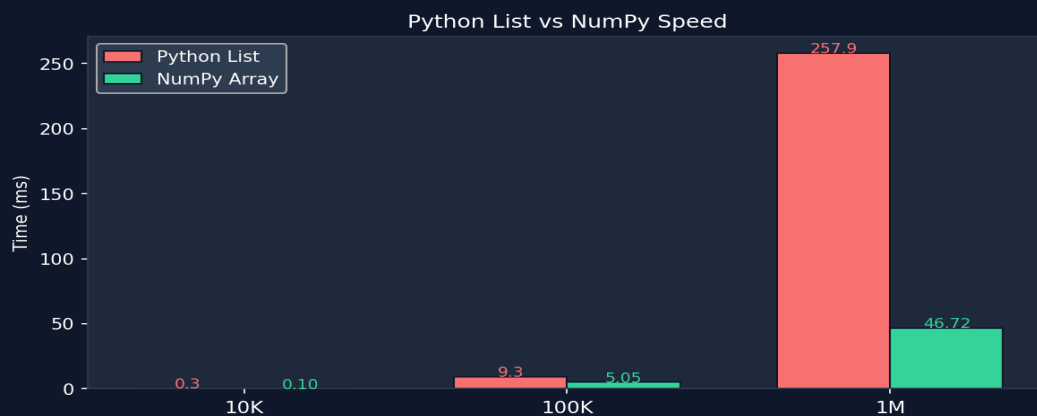


Fig 3 — NumPy vs Python list speed for element-wise multiplication across sizes.

Vectorisation Pattern

```

# BAD — Python loop (slow)
result = []
for x in data:
    result.append(x ** 2 + 2 * x + 1)

# GOOD — Vectorised (fast)
result = data**2 + 2*data + 1

# BAD — Manual dot product
dot = sum(a[i]*b[i] for i in range(len(a)))

# GOOD — NumPy
dot = np.dot(a, b)    # or a @ b

```

■ If you find yourself writing `np.vectorize()` to make a Python function work element-wise, it's still just a Python loop under the hood — nearly as slow as a for-loop. True vectorisation means expressing the operation with ufuncs and array arithmetic.

9. Common Gotchas & Edge Cases

Gotcha	What Happens	Fix
Integer division overflow	<code>np.int8(127) + 1 = -128</code> (wraps around)	Use <code>int32/int64</code> or check dtype
View vs Copy	Modifying a slice modifies the original	Use <code>.copy()</code> when you need independence
Shape (n,) vs (n,1)	Broadcasting behaves differently	Use <code>reshape(-1,1)</code> or <code>np.newaxis</code> explicitly
NaN propagation	Any op with NaN returns NaN silently	<code>np.nanmean()</code> , <code>np.isnan()</code> to detect
Floating point equality	<code>0.1 + 0.2 != 0.3</code> in float64	Use <code>np.isclose(a, b, atol=1e-8)</code>
In-place ops on views	<code>a += 1</code> on a slice affects original	Be explicit — understand when you have a view
<code>np.random.seed()</code> global state	Not thread-safe, affects all random calls	Use <code>np.random.default_rng(seed)</code> instead

NaN Handling

```
data = np.array([1.0, 2.0, np.nan, 4.0, np.nan])

np.isnan(data)           # [F F T F T]
data[~np.isnan(data)]    # [1. 2. 4.] — remove NaNs
np.nanmean(data)         # 2.333 — ignores NaNs
np.nansum(data)          # 7.0
np.nan_to_num(data, nan=0) # replace NaN with 0
```

10. Real-World ML Patterns with NumPy

Feature Normalisation (Z-score)

```
X = rng.normal(loc=50, scale=15, size=(1000, 4))

mean = X.mean(axis=0)      # per-feature mean, shape (4,)
std  = X.std(axis=0)       # per-feature std,  shape (4,)

X_norm = (X - mean) / std  # broadcasting handles shapes

# Verify
print(X_norm.mean(axis=0).round(8)) # ≈ [0. 0. 0. 0.]
print(X_norm.std(axis=0).round(8))  # ≈ [1. 1. 1. 1.]
```

Implementing Linear Regression Forward Pass

```
# X: (n_samples, n_features), W: (n_features,), b: scalar
def predict(X, W, b):
    return X @ W + b          # (n,f) @ (f,) → (n,)

# MSE loss
def mse(y_true, y_pred):
    return np.mean((y_true - y_pred)**2)

# Gradient of MSE w.r.t. W
def grad_W(X, y_true, y_pred):
    n = len(y_true)
    return (2/n) * X.T @ (y_pred - y_true) # (f,n)@(n,)→(f,)
```

Confusion Matrix from Scratch

```
def confusion_matrix(y_true, y_pred, n_classes):
    cm = np.zeros((n_classes, n_classes), dtype=int)
    for t, p in zip(y_true, y_pred):
        cm[t, p] += 1
    return cm

# Vectorised version (faster)
def confusion_matrix_v2(y_true, y_pred, n_classes):
    return np.bincount(n_classes * y_true + y_pred,
                       minlength=n_classes*2).reshape(n_classes, n_classes)
```

Softmax Function

```
def softmax(logits):
    # Subtract max for numerical stability (prevents overflow)
    shifted = logits - logits.max(axis=-1, keepdims=True)
    exp      = np.exp(shifted)
    return exp / exp.sum(axis=-1, keepdims=True)

logits = np.array([[2.0, 1.0, 0.1],
                   [1.0, 3.0, 0.2]])
probs = softmax(logits)    # rows sum to 1.0
```

Quick Reference Cheatsheet

Task	Code
Create from list	<code>np.array([1,2,3])</code>
Zeros / Ones	<code>np.zeros((m,n))</code> / <code>np.ones((m,n))</code>
Range	<code>np.arange(start, stop, step)</code>
Evenly spaced	<code>np.linspace(start, stop, num)</code>
Random normal	<code>np.random.default_rng(42).normal(0,1,(n,f))</code>
Shape / dtype	<code>arr.shape</code> / <code>arr.dtype</code>
Reshape	<code>arr.reshape(m, n)</code> or <code>arr.reshape(-1, 1)</code>
Transpose	<code>arr.T</code> or <code>arr.transpose(1,0)</code>
Slice	<code>arr[row_start:row_end, col_start:col_end]</code>
Boolean mask	<code>arr[arr > 0]</code>
Axis sum/mean	<code>arr.sum(axis=0)</code> / <code>arr.mean(axis=1)</code>
Matrix multiply	<code>A @ B</code> or <code>np.dot(A, B)</code>
Stack arrays	<code>np.vstack([a,b])</code> / <code>np.hstack([a,b])</code>
NaN safe mean	<code>np.nanmean(arr)</code>
Check NaN	<code>np.isnan(arr).any()</code>
Clip values	<code>np.clip(arr, min_val, max_val)</code>
Sort	<code>np.sort(arr)</code> / <code>np.argsort(arr)</code>
Unique	<code>np.unique(arr, return_counts=True)</code>
Save / Load	<code>np.save('f.npy', arr)</code> / <code>np.load('f.npy')</code>

Next Topic: Pandas — built directly on NumPy arrays, adding labelled axes, mixed-type columns, and powerful data manipulation tools for tabular data.